A Technical Overview

How the Psalms Commentary Pipeline Works

An AI pipeline for scholarly, verse-by-verse commentary on the תְּהִלִּים — how every agent and every source composes into the finished work.

What this system is

This is an AI pipeline that writes scholarly, verse-by-verse commentary on the Psalms — the kind of commentary you would hope to find from a gifted teacher who happens also to be a philologist, a liturgist, and a close reader of world literature. The output for a single psalm runs to tens of thousands of words: a governing introduction, a section on the psalm's life in Jewish liturgy, and a verse-by-verse treatment that quotes the Hebrew, defines its terms, traces its words across the Bible, weighs the medieval commentators, and occasionally reaches for Shakespeare or a Ugaritic hymn when the parallel genuinely illuminates.

A short excerpt gives the flavor better than a description. On Psalm 145's missing letter:

> *One letter is famously missing: nun (נ). The rabbis heard in that absence a dark echo — Amos 5:2's "Fallen (naflah), no more to rise, is the virgin Israel" — and then pointed to the very next verse as the correction: "The LORD supports all who fall" (v. 14). The gap itself becomes instruction.*

That single paragraph braids together an observation about poetic form, a rabbinic intertext, a Hebrew root, and a theological reading. Producing prose like that *reliably* — and keeping it accurate — is the whole engineering problem. No single model call does this well. Ask one model to "write a great commentary on Psalm 145," and you get fluent, confident, frequently wrong text: invented etymologies, misquoted verses, parallels that don't exist, and a breathless tone that mistakes adjectives for insight.

The architecture is a direct response to those failure modes. Its organizing principle is **separation of concerns**: break the work into specialized passes, let each pass do one thing well, ground every factual claim in a real source rather than in the model's memory, and only at the very end hand a single writer the job of turning all of it into prose. We sometimes call this *telescopic analysis* — each lens focuses on one layer, and the layers compose.

Two convictions run through the whole design:

1. **Discovery and writing are different cognitive tasks, and should not share a prompt.** The instinct to make a point well actively crowds out the patience to notice what's actually there. So the system discovers first, then writes.
2. **Facts come from databases and named sources; the model supplies judgment and voice, not data.** Hebrew text, concordance hits, lexicon entries, commentaries, liturgical placements, and statistical psalm-relationships are all retrieved deterministically. The model's job is to select, interpret, and articulate — not to recall.

The pipeline runs in roughly three movements: **analysis** (what is this psalm doing?), **research** (what does the rest of the canon, the lexicon, the tradition, and the scholarship say about it?), and **writing and quality control** (turn it into a single authorial voice, then verify every citation). What follows walks through each in order.

Movement I — Analysis

1. The Macro Analyst — the architect's pass

Model: Claude Opus 4.8, high reasoning effort.

The first pass reads the whole psalm (Hebrew and English) and produces a structural thesis: the genre, the historical and theological setting, an outline of the psalm's movement, its principal poetic devices (parallelism, chiasm, inclusio, sound-play), and — crucially — a set of five to ten *research questions* that the rest of the pipeline should try to answer. It writes its output in a deliberately terse, telegraphic notation, because this is an internal working document, not prose for the reader.

The Macro Analyst exists to give the project a spine before anyone looks at details. Without it, verse-by-verse analysis becomes a list of disconnected observations. With it, every later step knows what the poem is *about* and can recognize when a small detail serves (or subverts) the larger design.

2. The Micro Analyst — discovery, verse by verse

Model: Claude Sonnet 4.6, budgeted thinking.

The second pass reads each verse with what the prompt literally calls "fresh eyes," in **discovery mode rather than thesis mode**. Its instruction is to be curious, not conclusive — to find what is puzzling, surprising, or unusual, and explicitly to be willing to find that the macro thesis is *wrong*. It is fed the reconstructed phonetic transcription of each verse (see below), so it can notice sound patterns that are invisible on the page.

But the Micro Analyst's most important product is not its commentary — it is a structured set of **research requests**. As it reads, it decides what needs to be looked up:

- which rare or theologically loaded Hebrew words deserve a **lexicon** entry (it is told to skip common words like "day," "heart," "hand");
- which distinctive **roots** are worth tracing across the Bible, and which two-word **collocations** are worth a concordance search;
- which **images** to investigate in the figurative-language database;
- which verses are puzzling enough to warrant pulling the **traditional commentators**.

This is the hinge of the whole system. The model that decides *what to research* is doing genuine literary discernment; the research itself is then carried out deterministically by the "librarians" described in Movement II. The result is that research is *targeted* — driven by what the text actually raises — rather than a generic data dump.

A note on phonetics

A small deterministic component, the **Phonetic Analyst**, transcribes each verse into reconstructed Biblical Hebrew, with syllabification and stress marked (the stressed syllable rendered in capitals). It handles the things that matter for hearing a poem: *begadkefat* softening (b/v, k/kh, p/f), gemination from *dagesh forte*, vocal versus silent *shewa*, *maqqef* compounds as single stress domains, and divine-name modification. This is why the commentary can say that a line alliterates or that two words chime — and be right about it, rather than guessing from the consonantal spelling.

Movement II — Research: the librarians

Everything in this movement is **retrieval, not generation** (with one LLM-powered exception, the Figurative Curator, and the externally-prepared Literary Echoes and Deep Research files). These are Python agents that hit databases and APIs. They are deterministic, auditable, and free of model hallucination. Their results are assembled into a single large *research bundle* — up to several hundred thousand characters — which is later handed to the writer.

The word & phrase concordance

This is the system's intertextual engine, and it is more sophisticated than a keyword search. It runs against a concordance of the entire Hebrew Bible with a **four-layer normalization** scheme — exact (with cantillation), voweled, consonantal, and root — so that a search can be as strict or as forgiving as the question requires.

The decisive upgrade is **lemma-aware matching**. Every word in the concordance has been tagged with its dictionary lemma, drawn from the ETCBC/BHSA morphological database (the same scholarly morphology many digital-humanities projects rely on). This means a search for a root finds *all* its inflected, prefixed, and suffixed forms through a single indexed lookup — tolerant of conjugation, gender, prefix, and word order — without the brittle string- explosion that plagues surface-form search. A rare conjugated form is correctly recognized as belonging to a common lemma, and vice versa.

Two design choices make the results genuinely useful rather than noisy:

- **Distinctive-root selection.** Multi-word phrases lifted verbatim out of a verse almost never recur elsewhere; early versions of this tool found nothing but the source verse itself. So instead of searching whole phrases, the system picks the *single most distinctive root* in a lexical insight — distinctive meaning rare enough to be meaningful but common enough to recur (a frequency window, with particles and ultra-common verbs on a stop-list). Ranking is by true lemma frequency.
- **Self-match filtering with honest counts.** A search seeded from Psalm 67 filters out Psalm 67's own verses, and reports counts as "external" parallels versus "appears only in this psalm." The reader is never sold a phantom parallel.

The payoff is the discovery of non-obvious connections that neither a casual reader nor a model working from memory would surface: tracing a root for "separate/divide" to the Babel narrative, or a word for "wander" to Cain, or lining up "kindness and truth" with the divine self-revelation of Exodus 34:6. When the finished commentary says a word "is used elsewhere to mark human limits before divine wisdom (Job 5:9; 9:10; Isaiah 40:28)," those references came from here — and they are real, because they were looked up.

The figurative-language concordance ("Tzafun")

This is a distinctive and, I think, genuinely novel source. It is a separate research project — *Tzafun* (צָפַן, "hidden treasure") — that used a multi-model AI pipeline to build a **concordance of biblical figurative language**: every metaphor, simile, personification, idiom, hyperbole, and metonymy across the Torah, Psalms, Proverbs, and major prophets, each one detected, validated by a second model pass, and classified along four axes:

- **Target** — what is being described (e.g., *Judah*);
- **Vehicle** — what it is likened to (e.g., *a lion*);
- **Ground** — the shared quality (e.g., *strength*);
- **Posture** — the speaker's stance (celebratory, warning, and so on).

The result is a database of thousands of validated figurative instances, each with the AI's full deliberation recorded for transparency. The crucial property for commentary is that you can search it **conceptually rather than lexically**. The vehicle tags are concepts, not words: searching the vehicle "fortress" or "shepherd" finds every place in the corpus where God (or anyone) is *imaged* as one, regardless of the specific Hebrew word used. That lets the commentary ask the question a good reader actually wants answered — *why did the poet choose this image, and not the obvious alternative?* — and answer it with evidence about how the image behaves across Scripture.

Raw hits, though, are just rows. So a dedicated **Figurative Curator** (an LLM-driven agent with high reasoning) sits on top of the database. It runs an iterative search-and-refine loop — typically three rounds — examining the psalm's imagery, running vehicle searches, noticing gaps ("we have 'fortress' but not 'refuge'"), and re-searching with better terms until coverage is solid. It then curates five to fifteen of the best examples per image and writes four or five short interpretive notes that connect the corpus evidence to *this* psalm's choices. This is the one place in the research movement where a model adds interpretation rather than just data, and it is deliberately fenced off: its job is to turn the figurative concordance into something a writer can use.

The lexicon (BDB Librarian)

For the words the Micro Analyst flagged, this agent pulls full scholarly lexicon entries — **Brown-Driver-Briggs** and **Klein** — through Sefaria's API, including headword, Strong's number, morphology, semantic range, and (from Klein) etymology and derivatives. It disambiguates homographs by sense. This is how the commentary can say a verb "literally means to bubble up or gush (BDB)" with authority: the definition is the lexicon's, not the model's.

The division of labor here is principled and worth stating: **BDB defines what a word can mean; the concordance shows how the Bible actually uses it; the Figurative Curator explains why the poet chose it.** Three different sources, three different questions.

The traditional commentators (Commentary Librarian)

This agent fetches the classical Jewish *parshanut* for the requested verses, again through Sefaria, in Hebrew with English: **Rashi, Ibn Ezra, Radak, Metzudat David, Malbim, Meiri,** and **Torah Temimah** — a deliberate spread across the 11th to 20th centuries and across interpretive temperaments (*peshat*, grammar, philosophy, *midrash*). When the finished commentary writes "Ibn Ezra notes we exalt God 'by word and by the faithfulness of the heart,'" that is a real quotation, retrieved and then woven in.

Modern liturgical use (the liturgy pipeline)

Answering "where does this psalm live in actual Jewish prayer?" is harder than it sounds, and the system uses a **two-phase** approach.

- **Phase 0 (bootstrap):** Sefaria's own curated, hand-verified cross-references from Psalms into the liturgy give a fast, reliable first map.
- **Phase 4 (comprehensive):** A purpose-built *liturgy.db* indexes the liturgy at the **phrase level** across three traditions — **Ashkenaz, Sefard, and Edot HaMizrach** — recording, for each matched phrase, the prayer, the occasion (weekday / Shabbat / festival / High Holidays), the service (Shacharit / Mincha / Maariv / Musaf), and its position within the service. Matches carry confidence scores; full-psalm recitations are distinguished from individual verses and from stray phrases, and non-distinctive phrases are filtered out so that a common word doesn't generate spurious "liturgical" hits.

The raw matches are then turned into flowing scholarly prose by an LLM (Gemini 2.5 Pro primary, Claude Sonnet fallback), under a strict prompt: narrative, not lists; consolidate "Amidah (Ashkenaz)" and "Amidah (Sefard)" into "the Amidah across traditions"; include a real extended Hebrew quotation showing context; and carefully distinguish a phrase that sits *inside* a core prayer from one in a preliminary psalm *before* it. The summary is then quality-checked (length, narrative style, presence of a genuine Hebrew quotation) with automatic retry, and screened for the specific failure mode of attributing one psalm's usage to another.

This is what lets the introduction tell the reader exactly where they encounter the psalm in their own week, and — more interestingly — reflect on what the tradition's placement reveals about how it *understood* the psalm.

Statistically related psalms (Related Psalms Librarian)

A precomputed statistical analysis of all 150 psalms (the "V6" dataset) scores every psalm pair on three kinds of shared material: **shared roots** (weighted by IDF, so distinctive vocabulary counts more than common words), **contiguous exact phrases**, and **skipgrams** (gappy patterns — the same words in order with material in between). The librarian surfaces the top five relatives of the psalm in hand, with the actual shared Hebrew and verse contexts, and prompts the writer to look past mere word-overlap to shared

architecture, theme, and even instructive *differences*. This is how the commentary can place a psalm in a "diptych" with its statistical twin and mean it.

Rabbi Sacks (Sacks Librarian)

A curated corpus of Rabbi Jonathan Sacks' writing on the Psalms, indexed by reference with surrounding context. Sacks is included for a specific reason: he is not a classical *parshan* working on grammar, but a philosopher answering "why does this verse *matter*?" — a contemporary, ethically and philosophically engaged voice that complements the medievals.

Deep web research

For many psalms, a human has prepared a Gemini Deep Research dossier on the psalm's **afterlife** — its reception history, its role in art, music, and politics, scholarly debates, the surprising places it surfaces in cultural history (the Continental Congress, German *lieder*, African-American spirituals). This is loaded into the bundle and protected from trimming, because it is exactly the material a model cannot reliably produce from memory.

Cross-cultural literary echoes (the four-pass tool)

This is the source behind the commentary's occasional, carefully chosen reach into world literature. It is its own four-pass pipeline, and its design is a small case study in how the project fights hallucination:

1. **Generate** (Gemini 3.1 Pro): propose echoes between the psalm and the broad canon of world literature — Shakespeare, Dante, Homer, classical Chinese poetry, and beyond.
2. **Gap-fill** (Gemini 3.1 Pro): a second pass to find additional, less obvious voices the first pass missed.
3. **Verify** (GPT-5.4 *with live web search*): every proposed quotation is checked against the open web — does this line actually appear in this work? Is it accurately quoted? Is the attribution right? Fabrications and misquotes are flagged here, which is the entire point of the pass.
4. **Reconstruct** (GPT-5.4): rebuild a clean, publication-ready document that keeps the verified echoes and drops or demotes the doubtful ones.

A nice editorial touch: an **exclusion list** scans the most recently processed psalms and bans their authors from the current one, so that a reader working through the corpus doesn't meet Shakespeare in five consecutive psalms. The finished file enters the research bundle like any other source — but only after its quotations have survived being checked against reality.

Movement III — Writing and quality control

Cross-Verse Synthesis Discovery — the patience pass

Model: Claude Opus 4.8, high reasoning effort.

Before the writer writes, one more discovery pass runs over the whole dossier, looking for the patterns that only become visible when you hold *several verses together at once* — the kind of thing a verse-by-verse writer structurally cannot see, because writing each verse crowds out cross-verse noticing. It hunts for things like: a verb-tense pattern that tracks the psalm's emotional arc; two consonantally identical roots doing thematic double duty at opposite ends of the poem; an image the poem twice raises and twice retracts.

What makes this pass trustworthy is its **adversarial filtering**. Every candidate observation is attacked before it survives:

- a *novelty* filter ("would a scholar who knows Psalms already know this? then cut it"); and
- an *evidence-honesty* filter with nine explicit failure modes — among them: don't call two differently-vocalized words "the same word" (say "shares the consonants"); default to "echoes," never "verbatim," unless it is a literal contiguous match; don't stretch a word's meaning to land a point; every prooftext must be a real, contiguous, quotable substring; uniqueness claims ("the only place…") are almost always wrong; *count before you cite a number*.

There is no quota. If three observations survive, it outputs three; if eleven, eleven. Honesty about quantity is treated as part of the deliverable. The results are handed to the writer as *additional input, not instruction* — raw material to use where it serves the prose, not a structure to obey.

The Master Writer — the single authorial voice

Model: Claude Opus 4.8, adaptive thinking. (Configurable.)

This is the one step that produces reader-facing prose, and it produces *all* of it in one pass: the introduction, the liturgical section, and the verse-by-verse commentary. Everything upstream has been feeding this moment. The writer receives the whole dossier and is asked to write as a single scholar of the first rank — the prompt names Robert Alter, James Kugel, and Ellen Davis as the register — for an intelligent, Hebrew-literate, non-specialist reader.

The tone is specified precisely, and it is the opposite of typical AI prose:

> *Think scholar at dinner — relaxed, precise, occasionally witty in a dry and observational register, never performing. You don't need to prove you're smart; you need to make the psalm interesting.*

How the system actually *gets* that tone and that accuracy is a set of explicit, non-negotiable ground rules. The most consequential:

- **Hebrew and English always travel together**, and the translation is never a bare floating parenthetical — it is woven into the sentence. This single formatting discipline is what makes the prose feel like a book rather than an annotated dataset.
- **The Translation Test:** before keeping any observation, ask whether a reader could have figured it out from a good English translation alone. If yes, it's too obvious — cut it or go deeper. This is the rule that forces the commentary past the obvious.
- **No orphaned facts:** every philological or historical observation must immediately change the reader's understanding, or it gets cut. A routine grammatical form gets a routine sentence; only a transformative one gets a paragraph.
- **The blurry-photograph check:** banned vocabulary like *resonance, texture, dynamics, tapestry* — the words that simulate depth. The fix is concrete verbs: *what is God actually doing? what is the psalmist actually claiming?*
- **No false profundity:** the prompt specifically hunts the balanced, epigrammatic sentence that *sounds* wise but, stripped of its cadence, only restates a definition. Strip the rhythm; if no claim survives, it's decoration — cut it.
- **No jargon:** define every technical term on first use; never use words like *deixis, anaphora, telic, illocutionary*. Name the phenomenon, not its Latinate label.
- **Show, don't tell:** the words *masterpiece, tour de force, breathtaking, audacious* are forbidden. Demonstrate the brilliance instead.
- **Make connections explicit:** never just cite "(cf. Deut 33:28)" — explain *why* the cross-reference matters.
- **You are a scholar, not a pipeline endpoint:** the writer must never refer to "the thesis," "the research bundle," "the macro analysis," or "the insight extractor." It adopts good upstream insights *seamlessly, as its own*. The reader meets an author, not the seams of a system.

The wit is governed too — "dry, gentle, sparing," a side effect of seeing clearly rather than a performance. The prompt's gold-standard example ("Psalm 48 opens with the most extravagant real-estate listing in ancient literature") earns its keep precisely because it is *true*: the verse really does pile up four locational epithets.

The same writer runs in a **Special Instruction (SI)** mode, which is what produced the Psalm 67 files. A human supervisor can supply an overriding directive — emphasize a particular theme, foreground a particular reading, cut a particular kind of digression — which is inserted at highest priority into the writer's prompt without disturbing any of the standard machinery. The *_SI* outputs sit alongside the standard ones; nothing is overwritten. This is how the author keeps editorial control over the final character of a given psalm's commentary.

Copy Editor — the adversarial reader

Model: GPT-5.4, high reasoning effort.

A separate model now reads the finished draft as a skeptical editor, against a nine-category error taxonomy: it verifies **structural claims** (does the claimed chiasm actually have A-B-B'-A' order? does the inclusio really repeat?); catches **internal inconsistencies**; flags **form/content confusion** (claims that the sound "performs" the meaning when it doesn't); removes **negative citations** (a scholar named only to note his silence); checks **Hebrew script integrity**; tests **cross-cultural parallels** for real depth; verifies **factual and textual accuracy**; strips **grammatical bloat** (labels that add nothing); and challenges **strained arguments** — reversed causation, non sequiturs, overclaimed scope. It is told, for every paragraph, to identify the core claim and the evidence and ask whether a thoughtful first-time reader would actually be convinced. It preserves all formatting and divine-name conventions exactly, and emits a documented change-log.

Scripture Citation Verifier — zero-trust on quotations

Running just before the copy editor, this is a deterministic check with no model in the critical path. It extracts every Hebrew biblical quotation in the draft together with its citation, resolves the book name, pulls the *actual* verse from the Tanakh database, normalizes away vowels and cantillation, and checks that the quoted text is genuinely a substring of the real verse. Misquotes and non-existent citations are caught mechanically. A light GPT-5.1 pass then filters false positives (variant spellings, legitimate partial quotes), and any genuine problems are handed to the copy editor as a fix list. The effect is that a reader can trust that when the commentary quotes a verse and attributes it, the attribution is right.

Why the output is as rich — and as reliable — as it is

The richness on display in the Psalm 67 commentary is not the product of a single clever prompt. It is the accumulation of independent, grounded sources, each answering a different question, funneled to a writer who has been disciplined into a genuine authorial voice:

- the **lexicon** supplies the precise semantic range of a rare word;
- the **lemma-aware concordance** finds where else that word lives in the canon, turning a gloss into an intertext;
- the **figurative concordance (Tzafun)** explains why *this* image and not the obvious alternative, with corpus evidence behind it;
- the **commentators** supply nine centuries of *peshat*, grammar, and philosophy;
- the **liturgy pipeline** shows where the psalm lives in real prayer, across three traditions, and what that placement reveals;
- **related-psalms statistics** locate it among its kin;
- **Sacks**, **deep research**, and **verified literary echoes** open it onto modern thought, reception history, and world literature;
- the **synthesis-discovery** pass surfaces the cross-verse patterns the writer would otherwise miss.

And reliability is engineered in at every layer rather than hoped for at the end: facts are retrieved, not recalled; the literary echoes are web-verified; the synthesis pass is adversarially self-audited against nine named failure modes; the copy editor re-reads as a skeptic; and every Hebrew citation is checked character-by-character against the actual biblical text. The tone — precise, unshowy, occasionally witty, never breathless — is the product of a long list of explicit prohibitions whose common theme is: *say something true and concrete, or say nothing.*

The deepest design decision, and the one most responsible for the quality, is the refusal to let the model both invent the facts and write the prose. It writes the prose. The facts come from the library.

Models referenced are those configured at time of writing (Claude Opus 4.8, Claude Sonnet 4.6, GPT-5.1 / GPT-5.4, Gemini 3.1 Pro and Gemini 2.5 Pro); each agent's model is independently configurable, and the architecture — not any one model — is the point.